team_instructions

Modified Huffman Compression

Data Contract (Agreed File Structure)

To ensure seamless integration, the compressed binary file (`.bin`) MUST strictly follow this byte order:

1. **Overhead Length:** First `4 bytes` (Big Endian) representing the size of the JSON overhead.
2. **Overhead Data:** `N bytes` containing a UTF-8 encoded JSON string of the codes and 'Others' list.
3. **Padding Info:** `1 byte` integer (value 0-7) representing padding bits added to the end.
4. **Compressed Data:** `M bytes` of actual compressed bits.

Testing & Dummy Data Guide (START HERE)

We have generated 3 dummy files (find them in `/data` directory) for parallel testing:

1. `dummy_input.txt` (Text: "abaacaadaa xyz")
2. `dummy_huffman_tree.py` (Mock function returning hardcoded codes for the text)
3. `dummy_compressed.bin` (The correctly formatted binary result for the text)

Member 1: Algorithm, Tree & CLI (Main Wrapper)

Task: Build the Huffman tree and set up the `argparse` CLI.

- **Your Dummy Input:** Use `dummy_input.txt` to test your tree generation.
- **Your Expected Output:** Run your code on the text and ensure your output dictionary and 'Others' list look similar to the structure inside `dummy_huffman_tree.py` .
- **Next Step:** Build the `main()` function with flags (`-c` , `-d` , `-i` , `-o` , `-l`).

Member 2: Compression & Binary I/O

Task: Convert text to bits and save as a binary file based on the Data Contract.

- **Your Dummy Input:**
  1. Read `dummy_input.txt` .

2. Import and call the mock function from `dummy_huffman_tree.py` to get the codes (skip waiting for Member 1).

- **Your Expected Output:** Your code should generate a `.bin` file. Compare its size and structure to `dummy_compressed.bin` to ensure you applied the Data Contract correctly.

Member 3: Decompression & Integration

Task: Read the binary file, decompress it, and integrate everyone's code.

- **Your Dummy Input:** Use `dummy_compressed.bin` as your input file.
- **Your Expected Output:** Your decompression function must output a text file containing exactly `"abaacaadaa xyz"`. If it matches `dummy_input.txt`, your logic is 100% correct.
- **Next Step:** Wait for Member 1 and Member 2, then assemble their logic inside Member 1's CLI structure into the final `ModifiedHuffman_ID1_ID2_ID3.py` file.